INDEX

1. Abstract **2**
2. Introduction **2**
3. Aim and Objectives **2**
4. Requirements **3**
4.1 Software Requirements 3
4.2 Hardware Requirements 3
5. Theory **3**
5.1 Classification 3
5.2 Clinical Features Used 3
5.3 Random Forest Classifier 3
5.4 Train-Test Split and Stratification 3
5.5 Evaluation Metrics 4
6. Methodology **4**
7. Implementation: Code and Output **5**
7.1 Setup and Imports 5
7.2 Loading the Dataset 5
7.3 Training and Evaluating the Classifier 5
7.4 Visualising Model Performance 6
7.5 Predicting Remission for a Single Patient 7
7.6 Gradio Web Interface 8
7.7 Testing on New Patient Data 9
7.8 Patient Profiles for the Gradio Interface 11
7.9 Test Set Performance Summary 13
8. Results and Discussion **13**
8.1 Classifier Performance 13
8.2 Prediction on Single and New Patients 14
8.3 Performance Measures 14
8.4 Limitations 14
9. Conclusion **14**
10. Future Scope **14**
11. References **15**

1. Abstract

Rheumatoid arthritis (RA) is a long-term disease in which the immune system attacks the joints. Patients are often given biologic drugs called bDMARDs, but not every patient reaches clinical remission with them. If doctors could guess early which patient is likely to reach remission, the treatment could be planned better. In this project, a small machine learning pipeline is built in Python which predicts the binary outcome `remission` (1 = remission, 0 = no remission) from 13 clinical features of a patient.

The features are age, gender, BMI, four comorbidities (osteoarthritis, hypertension, diabetes and depression), swollen and tender joint counts (SJC28, TJC28), the pain score (VAS), the inflammation markers ESR and CRP, and the number of days the patient has taken bDMARD. A Random Forest classifier with 100 trees is trained on 80% of the data and tested on the remaining 20% (31 patients). The model reached 70.97% accuracy and a ROC AUC of 0.83 on the test set. A Gradio web interface was also made so that the values of a new patient can be typed in and the probability of remission is shown. This work is only for learning purpose and is not a medical device.

2. Introduction

Rheumatoid arthritis affects the joints and causes pain, swelling and stiffness. Treatment with bDMARDs (biologic disease-modifying anti-rheumatic drugs) is expensive and takes months to show its effect. The aim of the treatment is clinical remission, which means that the disease activity is very low. Some patients reach remission and some do not, and it is hard for a doctor to say in advance who will.

Machine learning is used here because many factors (age, inflammation, joint counts, other diseases) act together, and it is not easy to write them as simple rules. If we give the computer records of past patients along with their outcome, it can learn the pattern by itself. The overall process followed in this project is:

- Load the patient dataset and look at it.
- Select the useful clinical columns as features and `remission` as the target.
- Split the data into training and testing parts.
- Train a Random Forest classifier on the training part.
- Evaluate it using accuracy, classification report, confusion matrix and ROC curve.
- Use the model to predict for a single patient and for a batch of new patients.
- Make a simple web interface (Gradio) so that the model can be used without code.

3. Aim and Objectives

Aim

To design and implement a machine learning based classification model which predicts whether a rheumatoid arthritis patient will reach clinical remission, using baseline clinical features.

Objectives

- To load and explore the RA patient dataset (`data_bdmards.csv`).
- To select 13 clinical features and the binary target `remission`.

- To train a Random Forest classifier with a stratified 80/20 train-test split.
- To evaluate the model using accuracy, precision, recall, F1-score, confusion matrix and ROC-AUC.
- To write a function which predicts remission and its probability for one patient.
- To build a Gradio interface so that a non-programmer can use the model.
- To test the model on a new batch of patient records.

4. Requirements

4.1 Software Requirements

Item	Details
Language	Python 3
Platform	Google Colab (Jupyter Notebook)
Data handling	pandas, NumPy
Machine learning	scikit-learn
Plotting	matplotlib
Web interface	Gradio
Dataset	data_bdmards.csv (RA patients treated with bDMARD)

4.2 Hardware Requirements

A normal computer or laptop with internet connection is enough. Since the code runs on Google Colab, no GPU or special hardware is required.

5. Theory

5.1 Classification

Classification is a type of supervised learning in which the model learns from labelled examples and then predicts a class for new data. Here there are two classes, remission (1) and no remission (0), so it is a binary classification problem.

5.2 Clinical Features Used

- **Age, Gender, BMI:** basic details of the patient.
- **Comorbidities:** osteoarthritis, hypertension, diabetes and depression (0 = No, 1 = Yes).
- **SJC28 and TJC28:** number of swollen and tender joints out of the 28 joints examined. They are part of the standard DAS28 disease activity score.
- **VAS_pain:** pain score given by the patient on a scale of 0 to 100.
- **ESR and CRP:** blood tests which show the level of inflammation in the body. ESR is in mm/hr and CRP is in mg/L.
- **bDMARD_intake_duration_days:** number of days the patient has been taking the biologic drug.

5.3 Random Forest Classifier

Random Forest is an ensemble method which builds many decision trees on random parts of the data and takes the majority vote of all trees for the final answer (Breiman, 2001). It works well with a mix of different features, does not overfit easily, does not need feature scaling and also gives the probability of each class (the share of trees voting for it). Here 100 trees are used with random_state = 42.

5.4 Train-Test Split and Stratification

The data is divided into a training set (80%) and a test set (20%). The model learns only from the training set, and the test set is kept hidden to check how it behaves on new patients. Stratification keeps the ratio of remission and no-remission patients almost the same in both parts, so the test set is a fair sample.

5.5 Evaluation Metrics

- **Accuracy** = correct predictions / all predictions.
- **Precision** = TP / (TP + FP): out of all patients predicted as remission, how many really reached remission.
- **Recall** = TP / (TP + FN): out of all patients who really reached remission, how many were found.
- **F1-score** is the harmonic mean of precision and recall.
- **Confusion matrix** shows which classes are getting confused with which.
- **ROC curve and AUC:** the ROC curve plots the true positive rate against the false positive rate at different thresholds. AUC is the area under this curve; 0.5 means random guessing and 1.0 means perfect separation.

6. Methodology

The work was done in the following steps:

- **Step 1 - Load data:** the CSV file is read into a pandas DataFrame and a sample query (age above 30 and Gender = 1) is run to check the data.
- **Step 2 - Feature selection:** 13 clinical columns are chosen as X and `remission` as y. Outcome-like columns such as `sustained` and `effectiveness` are not used as inputs.
- **Step 3 - Split:** 80% training and 20% testing, stratified on y, random_state = 42.
- **Step 4 - Training:** a Random Forest with 100 trees is fitted on the training data.
- **Step 5 - Evaluation:** accuracy and classification report are printed; the confusion matrix and ROC curve are plotted.
- **Step 6 - Single patient prediction:** a function takes a dictionary of patient values and returns the class and its probability.
- **Step 7 - Gradio interface:** sliders, a dropdown and checkboxes are used to enter the patient values and to show the probabilities.
- **Step 8 - Batch testing:** five new (simulated) patient records are passed through the model and printed as readable profiles.
- **Step 9 - Summary:** the number of correct predictions in the test set is counted.

Important parameters used in the program are given below.

Parameter	Value	Meaning
test_size	0.2	20% of the data is kept for testing
stratify	y	Same class ratio in train and test sets
random_state	42	Fixed seed so that results can be repeated
n_estimators	100	Number of trees in the Random Forest
Features	13	Clinical input columns
Target	remission	0 = no remission, 1 = remission

7. Implementation: Code and Output

7.1 Setup and Imports

First the required libraries are imported.

```
import pandas as pd
import matplotlib.pyplot as plt
```

The libraries pandas and matplotlib are used for the data and the plots. The other libraries are imported later when they are needed.

7.2 Loading the Dataset

The dataset is read from the CSV file. If the file is not found, a small simulated dataset is created so that the cell can still run. A sample query is used to select the patients older than 30 years with Gender = 1.

```
import pandas as pd
import numpy as np


try:
    df = pd.read_csv('/content/data_bdmards.csv')
except FileNotFoundError:
    # File does not exist, so we simulate a dataset to allow the cell to run without error
    print("Warning: '/content/data_bdmards.csv' not found. Using simulated patient data instead.\n")
    np.random.seed(42)
    df = pd.DataFrame({
        'Age_years': np.random.randint(18, 80, size=50),
        'Gender': np.random.choice([0, 1], size=50),
        'BMI': np.random.uniform(18.5, 35.0, size=50),
        'remission': np.random.choice([0, 1], size=50)
    })


# Query patients older than 30 and with Gender == 1
result_df = df.query('Age_years > 30 and Gender == 1')
result_df
```

Output:

```
(Query result: 105 rows x 52 columns. The table is very wide, so only its size is shown here.)
```

The dataset has 52 columns, which include age, gender, comorbidities (osteoarthritis, asthma, uveitis, hypertension, COPD and others), drug details, RF and CCP antibody status, BMI and the outcome columns `sustained`, `remission` and `effectiveness`.

7.3 Training and Evaluating the Classifier

Thirteen columns are selected as features. The data is split into train and test sets (80% / 20%) and a Random Forest is trained. The accuracy and the classification report are shown.

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score
import pandas as pd


# 1. Select logical feature columns (excluding target or non-predictive ID columns)
feature_cols = [
```

```
    'Age_years', 'Gender', 'BMI',
    'Osteoarthritis', 'Hypertension', 'Diabetes', 'Depression',
    'SJC28', 'TJC28', 'VAS_pain',
    'ESR_mm', 'CRP_mg_l', 'bDMARD_intake_duration_days'
]


X = df[feature_cols]
y = df['remission']


# 2. Split the dataset into 80% train and 20% test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)


# 3. Initialize and train the Random Forest Classifier
model = RandomForestClassifier(random_state=42, n_estimators=100)
model.fit(X_train, y_train)


# 4. Make predictions and evaluate
y_pred = model.predict(X_test)


print(f"Model Accuracy: {accuracy_score(y_test, y_pred):.2%}\n")
print("Detailed Classification Report:")
print(classification_report(y_test, y_pred))
```

Output:

```
Model Accuracy: 70.97%


Detailed Classification Report:
              precision    recall  f1-score   support


         0.0       0.73      0.69      0.71        16
         1.0       0.69      0.73      0.71        15


    accuracy                           0.71        31
   macro avg       0.71      0.71      0.71        31
weighted avg       0.71      0.71      0.71        31
```

The test set has 31 patients: 16 without remission and 15 with remission, so the two classes are almost balanced.

7.4 Visualising Model Performance

To see how well the model separates remission from non-remission, a confusion matrix and a ROC curve are plotted.

```
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, roc_curve, auc


# Set up a side-by-side plot layout
fig, axes = plt.subplots(1, 2, figsize=(14, 5))


# Plot 1: Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['No Remission', 'Remission'])
disp.plot(ax=axes[0], cmap='Blues', values_format='d')
axes[0].set_title('Confusion Matrix')


# Plot 2: ROC Curve
y_score = model.predict_proba(X_test)[:, 1]
fpr, tpr, _ = roc_curve(y_test, y_score)
roc_auc = auc(fpr, tpr)


axes[1].plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC Curve (AUC = {roc_auc:.2f})')
axes[1].plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
axes[1].set_xlim([0.0, 1.0])
axes[1].set_ylim([0.0, 1.05])
axes[1].set_xlabel('False Positive Rate')
axes[1].set_ylabel('True Positive Rate')
axes[1].set_title('Receiver Operating Characteristic (ROC) Curve')
axes[1].legend(loc="lower right")
axes[1].grid(True, linestyle='--', alpha=0.5)


plt.tight_layout()
plt.show()
```

Output:

(Confusion matrix and ROC curve of the test set are shown below.)

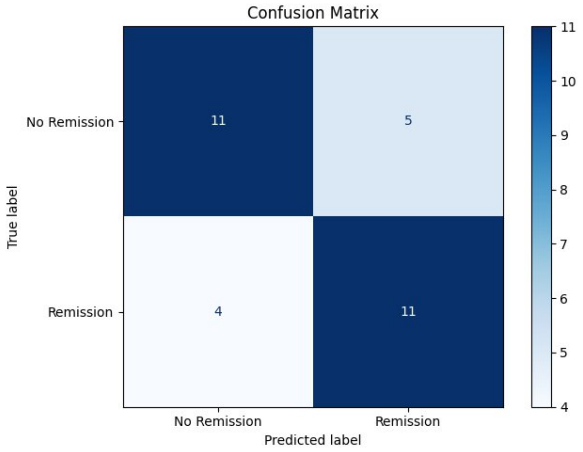


Figure 1: Confusion matrix of the Random Forest on the test set

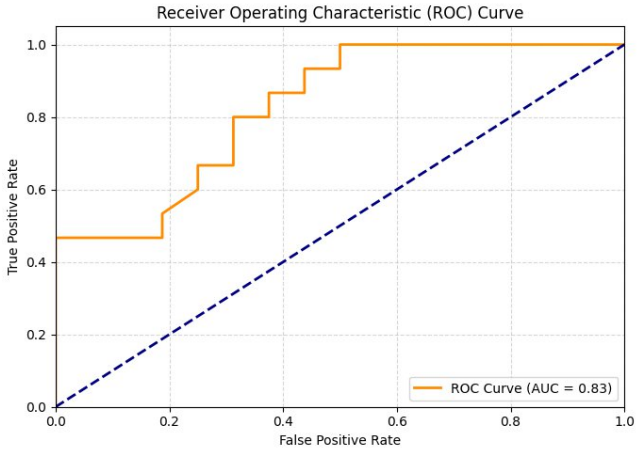


Figure 2: ROC curve of the Random Forest (AUC = 0.83)

In Figure 1, 11 patients without remission and 11 patients with remission were predicted correctly, while 5 patients without remission were wrongly predicted as remission and 4 remission patients were missed. In Figure 2 the ROC curve stays well above the dashed diagonal line (random guessing), which gives an AUC of 0.83.

7.5 Predicting Remission for a Single Patient

A function is written which takes the values of one patient as a dictionary, arranges them in the same column order as in training, and returns the prediction with its probabilities.

```
def predict_patient_remission(patient_data, trained_model, feature_names):
    """
    Predicts clinical remission for an individual patient.


    Parameters:
    - patient_data (dict): A dictionary containing values for each clinical feature.
    - trained_model: The trained Random Forest model.
    - feature_names (list): List of feature names used during training.
    """
    # Create a DataFrame from the single patient dictionary
    patient_df = pd.DataFrame([patient_data], columns=feature_names)


    # Make prediction and obtain probabilities
    prediction = trained_model.predict(patient_df)[0]
    probabilities = trained_model.predict_proba(patient_df)[0]


    # Output the results clearly
    print("--- Patient Report Prediction Results ---")
    if prediction == 1:
        print(f"Prediction: REMISSION (Probability: {probabilities[1]:.2%})")
    else:
        print(f"Prediction: NO REMISSION (Probability of Non-Remission: {probabilities[0]:.2%})")
    print("-----------------------------------------")
    return prediction, probabilities
```

```
# Example of how you would input a single patient's report:
example_patient = {
    'Age_years': 55.0,
    'Gender': 1.0,                      # e.g., 1 for Male / 0 for Female
    'BMI': 24.5,
    'Osteoarthritis': 0.0,              # 0 = No, 1 = Yes
    'Hypertension': 1.0,                # 0 = No, 1 = Yes
    'Diabetes': 0.0,                    # 0 = No, 1 = Yes
    'Depression': 0.0,                  # 0 = No, 1 = Yes
    'SJC28': 3.0,                       # Swollen Joint Count
    'TJC28': 4.0,                       # Tender Joint Count
    'VAS_pain': 45.0,                   # Visual Analog Scale of Pain (e.g. 0-100)
    'ESR_mm': 22.0,                     # Erythrocyte Sedimentation Rate
    'CRP_mg_l': 8.5,                    # C-Reactive Protein level
    'bDMARD_intake_duration_days': 120.0 # Treatment duration
}


# Run the prediction
pred, proba = predict_patient_remission(example_patient, model, feature_cols)
```

Output:

```
--- Patient Report Prediction Results ---
Prediction: NO REMISSION (Probability of Non-Remission: 89.00%)
-----------------------------------------
```

7.6 Gradio Web Interface

To make the model easy to use without writing code, a Gradio interface is created. It has sliders for the numbers (age, BMI, joint counts, pain, ESR, CRP, treatment days), a dropdown for gender and checkboxes for the four comorbidities. The probabilities of remission and no remission are shown as output.

```
import gradio as gr
import pandas as pd


def gradio_predict(
    age, gender, bmi, osteoarthritis, hypertension, diabetes, depression,
    sjc28, tjc28, vas_pain, esr, crp, duration
):
    # Map inputs to the structure expected by the model
    patient_data = {
        'Age_years': float(age),
        'Gender': 1.0 if gender == "Male" else 0.0,
        'BMI': float(bmi),
        'Osteoarthritis': 1.0 if osteoarthritis else 0.0,
        'Hypertension': 1.0 if hypertension else 0.0,
        'Diabetes': 1.0 if diabetes else 0.0,
        'Depression': 1.0 if depression else 0.0,
        'SJC28': float(sjc28),
        'TJC28': float(tjc28),
        'VAS_pain': float(vas_pain),
        'ESR_mm': float(esr),
        'CRP_mg_l': float(crp),
        'bDMARD_intake_duration_days': float(duration)
    }


    # Create a DataFrame from the patient data
    patient_df = pd.DataFrame([patient_data], columns=feature_cols)


    # Make prediction and get confidence percentages
    prob = model.predict_proba(patient_df)[0]


    return {
        "No Remission": float(prob[0]),
        "Remission": float(prob[1])
    }


# Design the Gradio user interface
inputs = [
    gr.Slider(minimum=18, maximum=100, value=55, label="Age (Years)"),
    gr.Dropdown(choices=["Female", "Male"], value="Male", label="Gender"),
    gr.Slider(minimum=15, maximum=45, value=24.5, label="BMI"),
    gr.Checkbox(label="Osteoarthritis Comorbidity"),
    gr.Checkbox(label="Hypertension Comorbidity"),
    gr.Checkbox(label="Diabetes Comorbidity"),
    gr.Checkbox(label="Depression Comorbidity"),
    gr.Slider(minimum=0, maximum=28, value=3, step=1, label="Swollen Joint Count (SJC28)"),
    gr.Slider(minimum=0, maximum=28, value=4, step=1, label="Tender Joint Count (TJC28)"),
    gr.Slider(minimum=0, maximum=100, value=45, label="Patient Pain VAS Score (0-100)"),
    gr.Slider(minimum=0, maximum=120, value=22, label="ESR (mm/hr)"),
    gr.Slider(minimum=0.0, maximum=100.0, value=8.5, label="CRP (mg/L)"),
    gr.Slider(minimum=0, maximum=730, value=120, label="bDMARD Treatment Duration (Days)")
]
```

```
# Create the interface dashboard
demo = gr.Interface(
    fn=gradio_predict,
    inputs=inputs,
    outputs=gr.Label(num_top_classes=2, label="Remission Prediction Probability"),
    title="Rheumatoid Arthritis Clinical Remission Predictor",
    description="Input patient report parameters below to predict the probability of reaching clinical
remission using the trained machine learning model."
)


# Launch the interface with sharing turned on
demo.launch(share=True)
```

Output:

```
Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://d20b3b9fc1dc504254.gradio.live


This share link is temporary and will last for up to 1 week (best effort). For free permanent hosting and GPU
upgrades, run `gradio deploy` from the terminal in the working directory to deploy to Hugging Face Spaces
(https://huggingface.co/spaces)
(The Gradio interface opens as an interactive web page inside the notebook, so it cannot be shown here.)
```

7.7 Testing on New Patient Data

To test the whole prediction flow on data not used in training, a batch of 5 new patient records is generated with random values in realistic ranges (a simulated batch, not real patients). The records are passed through the model and the predicted class and probability of remission are attached to the table.

```
import pandas as pd
import numpy as np


# Since hosting external files dynamically can sometimes lead to 404 errors,
# let's generate a new, simulated batch of patient records based on your original dataset
# to fully demonstrate the prediction testing flow.


try:
    # Simulate a set of 5 new patients received from an external clinic
    np.random.seed(42)
    new_data_dict = {
        'Age_years': np.random.randint(30, 80, size=5).astype(float),
        'Gender': np.random.choice([0.0, 1.0], size=5),
        'BMI': np.round(np.random.uniform(18.5, 35.0, size=5), 1),
        'Osteoarthritis': np.random.choice([0.0, 1.0], size=5),
        'Hypertension': np.random.choice([0.0, 1.0], size=5),
        'Diabetes': np.random.choice([0.0, 1.0], size=5),
        'Depression': np.random.choice([0.0, 1.0], size=5),
        'SJC28': np.random.randint(0, 15, size=5).astype(float),
        'TJC28': np.random.randint(0, 15, size=5).astype(float),
        'VAS_pain': np.random.randint(10, 90, size=5).astype(float),
        'ESR_mm': np.random.randint(5, 60, size=5).astype(float),
        'CRP_mg_l': np.round(np.random.uniform(1.0, 30.0, size=5), 1),
        'bDMARD_intake_duration_days': np.random.randint(30, 365, size=5).astype(float)
    }


    new_patients_df = pd.DataFrame(new_data_dict)
    print("New Patient Batch loaded successfully! Here is the input data:")
    display(new_patients_df)


    # Prepare features for prediction
    X_new = new_patients_df[feature_cols]


    # Generate predictions and probabilities
    predictions = model.predict(X_new)
    probabilities = model.predict_proba(X_new)[:, 1]


    # Attach results to the DataFrame
    new_patients_df['Predicted_Remission'] = predictions
    new_patients_df['Remission_Probability'] = probabilities


    print("\n--- External Batch Predictions Completed ---")
    display(new_patients_df[['Age_years', 'SJC28', 'TJC28', 'CRP_mg_l', 'Predicted_Remission',
'Remission_Probability']])


except Exception as e:
    print(f"An error occurred while preparing or predicting new data: {e}")
```

Output:

```
New Patient Batch loaded successfully! Here is the input data:


   Age_years  Gender   BMI  Osteoarthritis  Hypertension  Diabetes  \
0       68.0     0.0  20.1             0.0           1.0       0.0
```

```
1       58.0     0.0  26.1             1.0           1.0       1.0
2       44.0     0.0  24.0             1.0           1.0       1.0
3       72.0     1.0  20.9             1.0           1.0       1.0
4       37.0     0.0  29.2             1.0           0.0       0.0


   Depression  SJC28  TJC28  VAS_pain  ESR_mm  CRP_mg_l  \
0         1.0   14.0   13.0      64.0    25.0      28.3
1         0.0    9.0   13.0      73.0    13.0      17.3
2         0.0   11.0   14.0      12.0    43.0      12.2
3         0.0   11.0   13.0      60.0    22.0       1.5
4         0.0   14.0    2.0      16.0     8.0       7.7


   bDMARD_intake_duration_days
0                        121.0
1                        293.0
2                         64.0
3                        235.0
4                        110.0


--- External Batch Predictions Completed ---


   Age_years  SJC28  TJC28  CRP_mg_l  Predicted_Remission  \
0       68.0   14.0   13.0      28.3                  0.0
1       58.0    9.0   13.0      17.3                  0.0
2       44.0   11.0   14.0      12.2                  0.0
3       72.0   11.0   13.0       1.5                  0.0
4       37.0   14.0    2.0       7.7                  0.0


   Remission_Probability
0                   0.18
1                   0.21
2                   0.32
3                   0.15
4                   0.41
```

7.8 Patient Profiles for the Gradio Interface

Each patient of the new batch is printed in a readable format so that the values can be copied into the sliders of the Gradio interface. The model confidence is also shown.

```
# Print out each patient profile from the simulated new batch in a human-readable format
for i, patient in new_patients_df.iterrows():
    print(f"=== Patient Profile #{i+1} ===")
    print(f"Age: {patient['Age_years']}")
    print(f"Gender: {'Male' if patient['Gender'] == 1.0 else 'Female'}")
    print(f"BMI: {patient['BMI']}")
    print(f"Osteoarthritis: {bool(patient['Osteoarthritis'])}")
    print(f"Hypertension: {bool(patient['Hypertension'])}")
    print(f"Diabetes: {bool(patient['Diabetes'])}")
    print(f"Depression: {bool(patient['Depression'])}")
    print(f"SJC28 (Swollen Joint Count): {patient['SJC28']}")
    print(f"TJC28 (Tender Joint Count): {patient['TJC28']}")
    print(f"VAS Pain Score: {patient['VAS_pain']}")
    print(f"ESR (mm/hr): {patient['ESR_mm']}")
    print(f"CRP (mg/L): {patient['CRP_mg_l']}")
    print(f"Treatment Duration (Days): {patient['bDMARD_intake_duration_days']}")
    print(f"--> Model Prediction: {'REMISSION' if patient['Predicted_Remission'] == 1.0 else 'NO REMISSION'}
(Confidence: {max(patient['Remission_Probability'], 1 - patient['Remission_Probability']):.2%})")
    print("="*30 + "\n")
```

Output:

```
=== Patient Profile #1 ===
Age: 68.0
Gender: Female
BMI: 20.1
Osteoarthritis: False
Hypertension: True
Diabetes: False
Depression: True
SJC28 (Swollen Joint Count): 14.0
TJC28 (Tender Joint Count): 13.0
VAS Pain Score: 64.0
ESR (mm/hr): 25.0
CRP (mg/L): 28.3
Treatment Duration (Days): 121.0
--> Model Prediction: NO REMISSION (Confidence: 82.00%)
==============================


=== Patient Profile #2 ===
Age: 58.0
Gender: Female
BMI: 26.1
Osteoarthritis: True
Hypertension: True
Diabetes: True
Depression: False
SJC28 (Swollen Joint Count): 9.0
TJC28 (Tender Joint Count): 13.0
VAS Pain Score: 73.0
```

```
ESR (mm/hr): 13.0
CRP (mg/L): 17.3
Treatment Duration (Days): 293.0
--> Model Prediction: NO REMISSION (Confidence: 79.00%)
==============================


=== Patient Profile #3 ===
Age: 44.0
Gender: Female
BMI: 24.0
Osteoarthritis: True
Hypertension: True
Diabetes: True
Depression: False
SJC28 (Swollen Joint Count): 11.0
TJC28 (Tender Joint Count): 14.0
VAS Pain Score: 12.0
ESR (mm/hr): 43.0
CRP (mg/L): 12.2
Treatment Duration (Days): 64.0
--> Model Prediction: NO REMISSION (Confidence: 68.00%)
==============================


=== Patient Profile #4 ===
Age: 72.0
Gender: Male
BMI: 20.9
Osteoarthritis: True
Hypertension: True
Diabetes: True
Depression: False
SJC28 (Swollen Joint Count): 11.0
TJC28 (Tender Joint Count): 13.0
VAS Pain Score: 60.0
ESR (mm/hr): 22.0
CRP (mg/L): 1.5
Treatment Duration (Days): 235.0
--> Model Prediction: NO REMISSION (Confidence: 85.00%)
==============================


=== Patient Profile #5 ===
Age: 37.0
Gender: Female
BMI: 29.2
Osteoarthritis: True
Hypertension: False
Diabetes: False
Depression: False
SJC28 (Swollen Joint Count): 14.0
TJC28 (Tender Joint Count): 2.0
VAS Pain Score: 16.0
ESR (mm/hr): 8.0
CRP (mg/L): 7.7
Treatment Duration (Days): 110.0
--> Model Prediction: NO REMISSION (Confidence: 59.00%)
==============================
```

7.9 Test Set Performance Summary

Finally the actual and predicted outcomes of the test set are compared and the correct predictions are counted.

```
import pandas as pd


# Create a summary DataFrame of our actual test set vs predicted outcomes
evaluation_df = pd.DataFrame({
    'Actual_Remission': y_test,
    'Predicted_Remission': y_pred
})


# Calculate counts
total_patients = len(evaluation_df)
actual_remission_count = int((evaluation_df['Actual_Remission'] == 1).sum())
correct_predictions = int((evaluation_df['Actual_Remission'] == evaluation_df['Predicted_Remission']).sum())


# True Positives: Patients who actually achieved remission AND were predicted correctly
correctly_predicted_remission = int(((evaluation_df['Actual_Remission'] == 1) &
(evaluation_df['Predicted_Remission'] == 1)).sum())


# True Negatives: Patients who did not achieve remission AND were predicted correctly
correctly_predicted_non_remission = int(((evaluation_df['Actual_Remission'] == 0) &
(evaluation_df['Predicted_Remission'] == 0)).sum())


print(f"--- Test Set Performance Summary ({total_patients} Patients) ---")
print(f"Total Patients who actually achieved Remission (Cured): {actual_remission_count}")
print(f"Total Predictions that were CORRECT: {correct_predictions} out of {total_patients}
({correct_predictions/total_patients:.2%})")
print(f"  - Correctly predicted as REMISSION: {correctly_predicted_remission}")
print(f"  - Correctly predicted as NO REMISSION: {correctly_predicted_non_remission}")
```

```
print("--------------------------------------------------")
```

Output:

```
--- Test Set Performance Summary (31 Patients) ---
Total Patients who actually achieved Remission (Cured): 15
Total Predictions that were CORRECT: 22 out of 31 (70.97%)
  - Correctly predicted as REMISSION: 11
  - Correctly predicted as NO REMISSION: 11
--------------------------------------------------
```

8. Results and Discussion

8.1 Classifier Performance

Class	Precision	Recall	F1-score	Support
No remission (0)	0.73	0.69	0.71	16
Remission (1)	0.69	0.73	0.71	15
Accuracy			0.71	31

The model got 70.97% accuracy on the 31 test patients, which means 22 patients were classified correctly. From the confusion matrix (Figure 1), 11 patients with remission and 11 patients without remission were found correctly, 5 patients without remission were wrongly predicted as remission (false positives) and 4 patients with remission were missed (false negatives). The precision and recall are almost the same for both classes, so the model is not simply favouring one class.

8.2 Prediction on Single and New Patients

For the example patient (55 years, SJC28 = 3, TJC28 = 4, CRP = 8.5 mg/L) the model predicted NO REMISSION with 89% probability. On the batch of 5 simulated patients, all were predicted as no remission, with remission probabilities of 0.18, 0.21, 0.32, 0.15 and 0.41. These 5 records were random values and not real patients, so this result only shows that the prediction pipeline and the Gradio interface work. It does not tell anything about the real accuracy.

8.3 Performance Measures

Measure	Value	Meaning
Accuracy	70.97% (22/31)	Correct predictions out of all test patients
Sensitivity (recall of remission)	73.3% (11/15)	Remission patients correctly found
Specificity	68.8% (11/16)	Non-remission patients correctly found
ROC AUC	0.83	Chance that a remission patient is ranked above a non-remission patient

The AUC of 0.83 is higher than the accuracy of 71%, which shows that the probabilities given by the model carry more information than the default 0.5 cut-off shows. The AUC is also clearly better than random guessing (0.5).

8.4 Limitations

- The test set has only 31 patients, so one patient changes the accuracy by about 3 percentage points. The 70.97% value can change a lot with another split.
- Only a single train-test split was used. No cross-validation or hyperparameter tuning was done.

- Only 13 of the 52 columns were used, and the model was not checked on data from another hospital.
- The batch of 5 new patients was simulated, so it is not a real test.
- The coding of the Gender column (0/1) should be checked against the data dictionary of the dataset before any conclusion is drawn from it.
- This is not a medical device and cannot be used for diagnosis or for deciding treatment.

9. Conclusion

In this project a machine learning model was built to predict clinical remission in rheumatoid arthritis patients from 13 baseline clinical features. A Random Forest classifier gave 70.97% accuracy, balanced precision and recall of about 0.71 for both classes, and a ROC AUC of 0.83 on the test set. A prediction function, a Gradio web interface and a batch testing step were also made, so the complete flow from data to a usable tool is covered.

From this work I understood how a clinical dataset is prepared for classification, how a stratified split and a Random Forest are used, how the results are read through a confusion matrix and a ROC curve, and how a trained model can be given to users through a simple interface. The accuracy is moderate because the dataset is small, but the basic idea works and can improve with more data.

10. Future Scope

- Use a larger multi-centre RA dataset and keep a separate external test set.
- Use cross-validation and hyperparameter tuning (for example GridSearchCV) to get stable results.
- Compare other models such as Logistic Regression, Gradient Boosting, XGBoost and SVM.
- Use more of the 52 columns and study the feature importance to know which factors matter most.
- Handle class imbalance and missing values more carefully and check the calibration of the probabilities.
- Use explainability tools such as SHAP so that doctors can see why a prediction was made.
- Host the interface permanently (for example on Hugging Face Spaces) and test it with doctors.

11. References

[1] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, 2001, pp. 5-32.

[2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, 2011, pp. 2825-2830.

[3] T. Fawcett, "An introduction to ROC analysis," Pattern Recognition Letters, vol. 27, no. 8, 2006, pp. 861-874.

[4] M. L. L. Prevoo et al., "Modified disease activity scores that include twenty-eight-joint counts," Arthritis & Rheumatism, vol. 38, no. 1, 1995, pp. 44-48.

[5] W. McKinney, "Data Structures for Statistical Computing in Python," Proceedings of the 9th Python in Science Conference, 2010, pp. 56-61.

[6] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," Computing in Science & Engineering, vol. 9, no. 3, 2007, pp. 90-95.

[7] A. Abid et al., "Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild," arXiv:1906.02569, 2019.

[8] Dataset: data_bdmards.csv (RA patients treated with bDMARD) - [add source of the dataset here].